NLP
ASSIGNMENT 1
Name: JIYA SINHA
Roll no: 22161

TEXT CLASSIFICATION USING FFNN

Steps followed:

- **Loading and preprocessing:** After loading the csv file and dropping any rows with missing values, the following preprocessing steps are used:
  - Converting it to lowercase
  - Removing the URLs
  - Change multiple dots to single dot
  - Remove extra white spaces
  - Remove non-alphabetical characters.
- **Form Vocabulary**:
  Form a vocabulary (unique words in the training data) and form dictionaries for mapping words to unique index (word2index) and vice versa (index2word).
- **Skip Gram:**
  Form a list of target words, a dictionary mappingeach target word to the positive context words (the words neighboring the target word, within a window), and a dictionary mapping each target word to k negative words (words not near the target word, i.e., outside the target window).

  Example:
  For a window size = 4, k = 5:
  Target word: maarliek
  Its positive samples are: ['ne', 'k', 'ki', 'mohenjodaro', 'sath', 'bhi', 'hritik', 'hadonone']
  Its negative samples are: ['agar', 'lost', 'ajeebogareeb', 'ba', 'bana']

- **Form training data for creating embeddings**: Create a feature vector $X = (targetword, contextword)$ and a corresponding label vector $y \in \{0, 1\}$, where 0 denotes a negative context word and 1 denotes a positive context word.
  Since the sampling for negative words is done randomly, different sessions may generate different negative context words.
  The feature vector (converted to words for visualization purposes, for training the indices are used) is:

```
array([['maarliek', 'bhi'],
       ['maarliek', 'hadonone'],
       ['maarliek', 'k'],...
       ['maarliek', 'ajeebogareeb'],
       ['maarliek', 'ba'],
       ['maarliek', 'bana']], dtype=object)
```

- Retrieve target_words and context_words from X_train.
- Create two input layers (target_ip and context_ip) for target and context words.
- Randomly initialize embeddings for target and context words of size vocab_size x 100
- Use dot product between the target and context word embeddings to measure similarity.
- Use sigmoid activation function to predict using word pair similiarity.
- Use the Adam optimizer and binary cross-entropy loss.
- Trains the model with given epochs and batch_size.
- Retrieve the learned target and context embeddings from the trained model.
- Average the target and context embeddings to form the final word embeddings and save it.

- **Prepare dataset for implementing FFNN:**
  Extracted X_train, y_train, X_val and y_val, converted the words in the sentence to word index and make all the sentences of same size = max of max of the length of sentence in X_train and X_val by padding.

- **Training FFNN:**
  - Load word embeddings from a file and determine the embedding_dim = embedding shape
  - Initialize and define the model:
    * Embedding layer: Initializes an embedding layer using pretrained embeddings, with mask_zero=True for handling padding.
    * Global average pooling layer: Reduces dimensionality by averaging word embeddings.
    * Dense layer: A fully connected layer with 64 neurons and ReLU activation.
    * Output layer: A dense layer with a sigmoid activation function for binary classification.
  - Use adam optimizer, binary cross entropy loss as the loss function and track accuracy as a performance metric.
  - Train the model given a number of epochs and batch size. Also handle class imbalance using keras compute_class_weight().

- **Analysis:**
  Table 1 summarizes the classification performance across different tasks.

	F1-score	
Dataset/Tags	0	1
Hate	0.72	0.49
Humor	0.51	0.73
Sarcasm	0.98	0.77

Table 1: F1-Scores